Tufte's Principles of Graphical Excellence

Workshop 1 — Module 2

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. The data-ink ratio: theory and application
2. Chartjunk: a taxonomy of visual noise
3. The lie factor: graphical integrity
4. Small multiples and sparklines
5. Integration of text, numbers, and graphics

Learning Outcome
You will be able to evaluate any chart against Tufte's core principles and systematically improve its data-ink ratio.

The Data-Ink Ratio

Tufte's Central Principle

```
Data-Ink Ratio  =  Ink used to represent data  /  Total ink on graphic
```

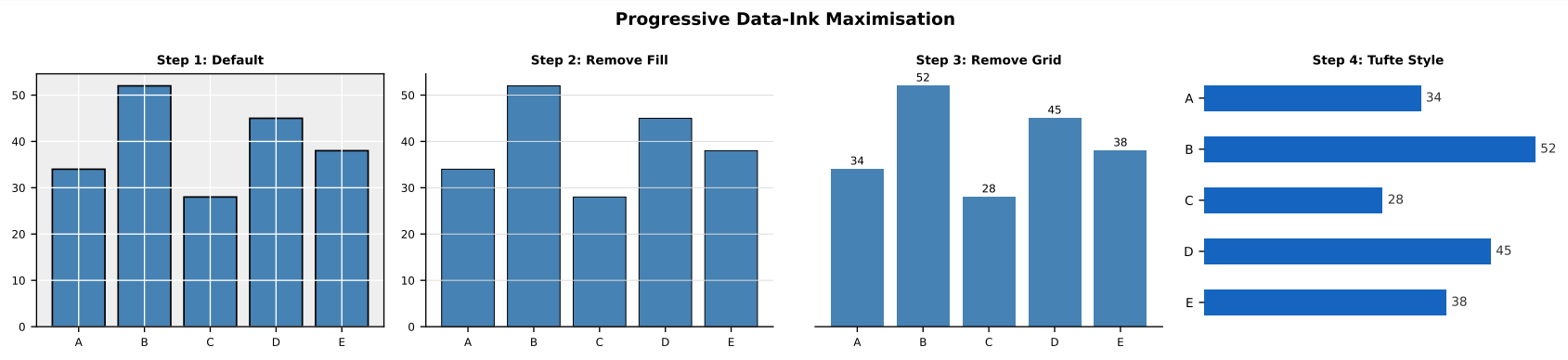
Goal: maximise this ratio towards 1.0
"Erase non-data ink, within reason" — Tufte (1983)

Two operational corollaries:

1. Above all else, **show the data**.
2. Erase non-data ink, within reason.

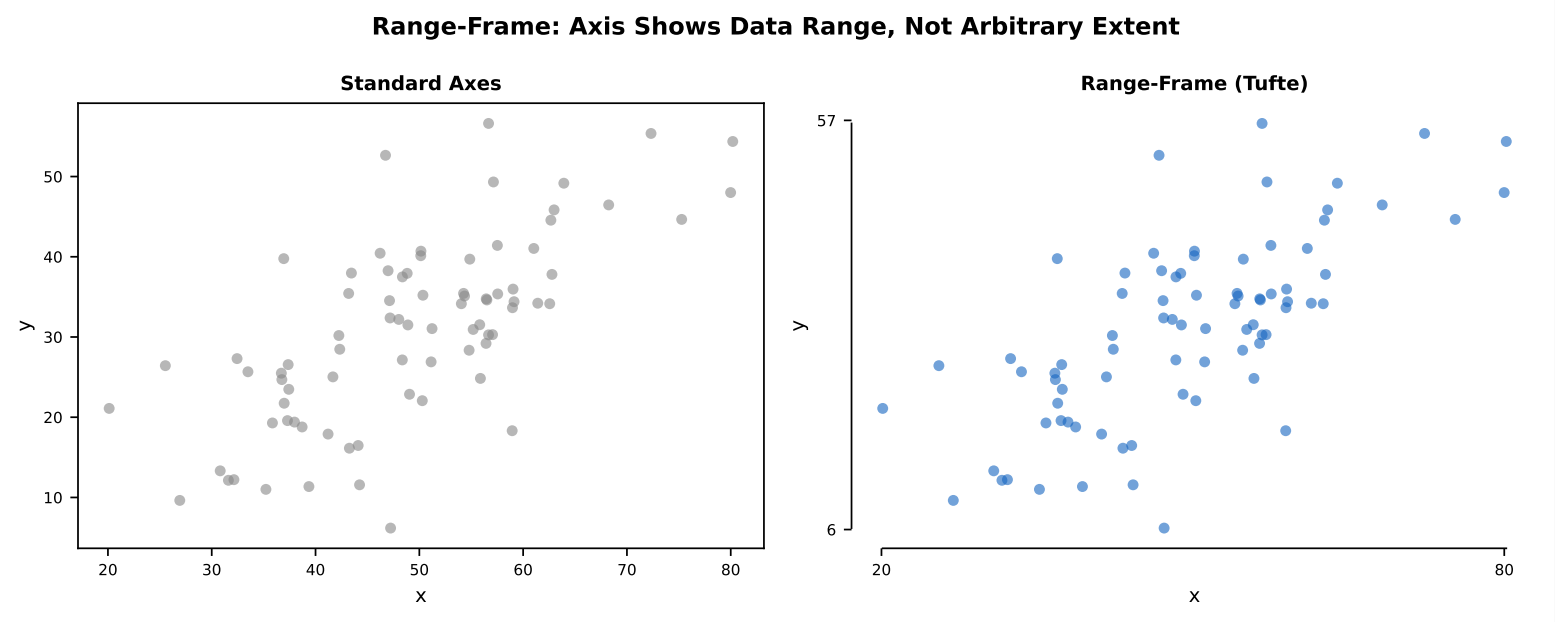
Source: Tufte, E. R. (1983). The Visual Display of Quantitative Information, pp. 93–96.

Progressive Data-Ink Maximisation



Pro-Tip
Apply the "eraser" iteratively: remove one element, check if anything is lost. If not, it was non-data ink.

The Range-Frame: Tufte's Axis Innovation

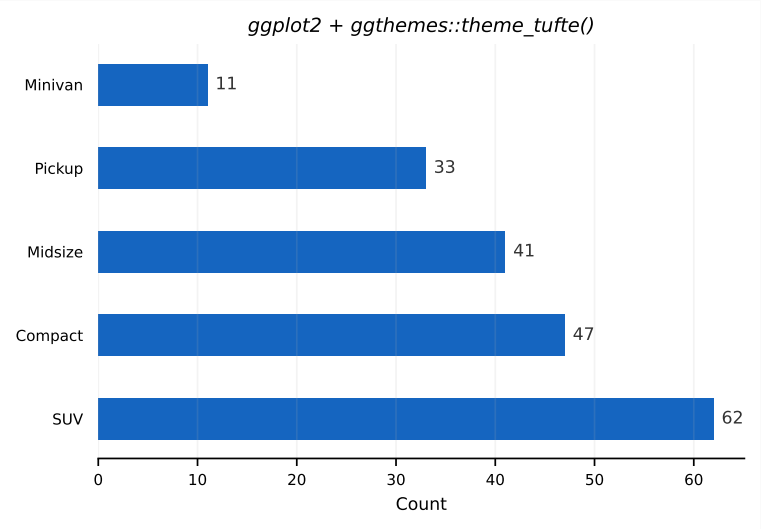


Data Insight

In a range-frame the axis line extends only from the data minimum to the data maximum. The axis itself becomes a data display showing range, not an arbitrary container.

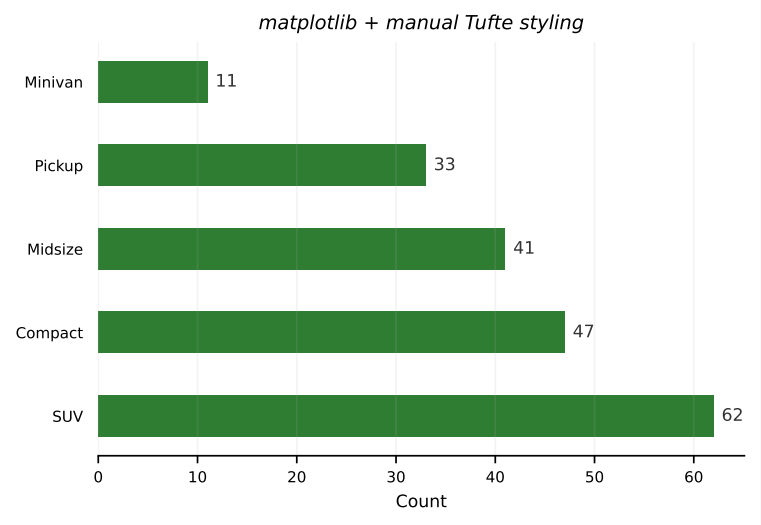
Data-Ink Reduction in R

```r
library(ggplot2)
library(ggthemes) # theme_tufte()

# Standard ggplot bar chart
p <- ggplot(mpg_counts,
       aes(x = fct_reorder(class, n
            ),
           y = n)) +
  geom_col(fill = "#1565C0",
           width = 0.6) +
  geom_text(aes(label = n),
            hjust = -0.2,
            size = 3) +
  coord_flip() +
  # Apply Tufte theme: removes
  # gridlines, axis ticks, borders
  theme_tufte(base_size = 11) +
  labs(x = NULL, y = "Count",
       title = "Vehicle Class")
```



Data-Ink Reduction in Python

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.barh(cats, vals,
        color="#2E7D32",
        height=0.5)

# Direct labels      no legend
    needed
for i, v in enumerate(vals):
    ax.text(v + 0.8, i, str(v),
            va="center", fontsize
                =8)

# Tufte-style spine removal
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
    False)
ax.spines["left"].set_visible(False
    )
ax.tick_params(left=False)
ax.grid(axis="x", alpha=0.15)

ax.set_xlabel("Count")
plt.tight_layout()
```



Chartjunk

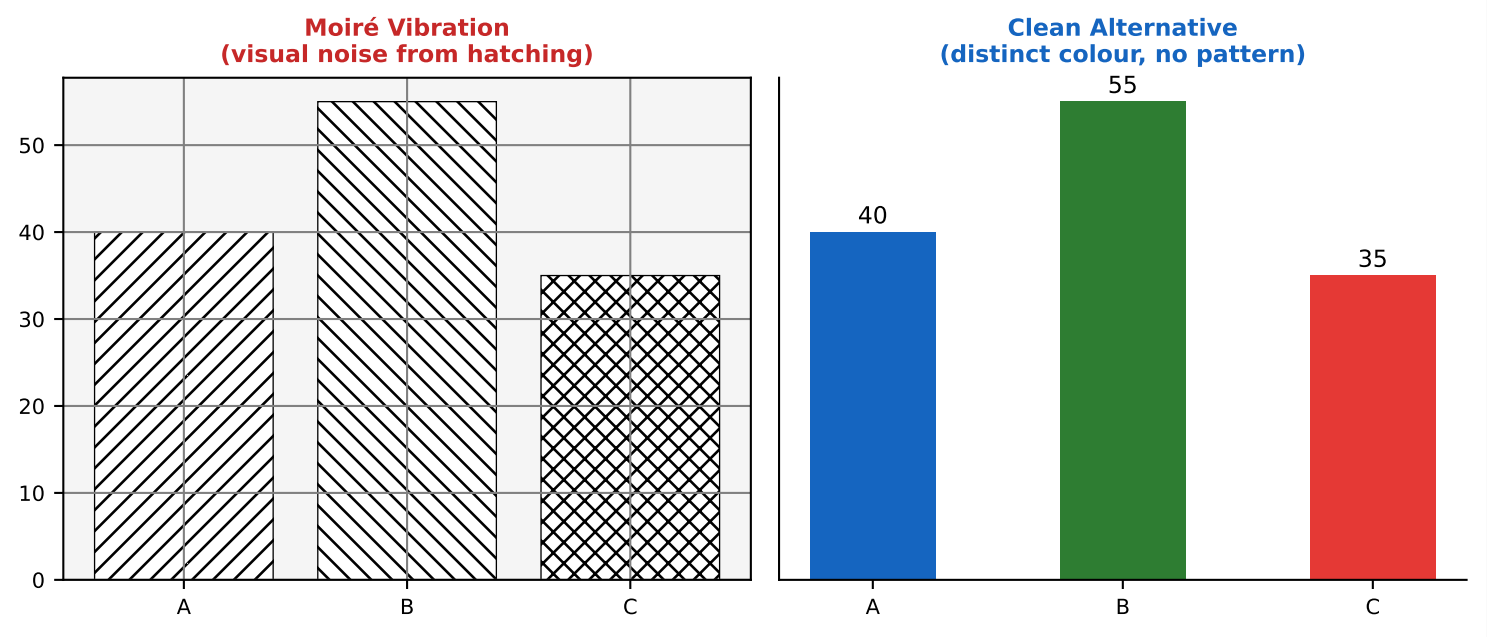
Tufte's Chartjunk Taxonomy

Chartjunk consists of visual elements that do not convey data:

1. **Moiré vibration** — dense hatching patterns that flicker and distract the eye
2. **The grid** — heavy gridlines that compete with data marks for visual attention
3. **The duck** — decorative images and clip-art that overwhelm the actual data

"Chartjunk does not achieve the goals of its propagators. The overwhelming fact of data graphics is that they stand or fall on their content." — Tufte (1983), p. 121

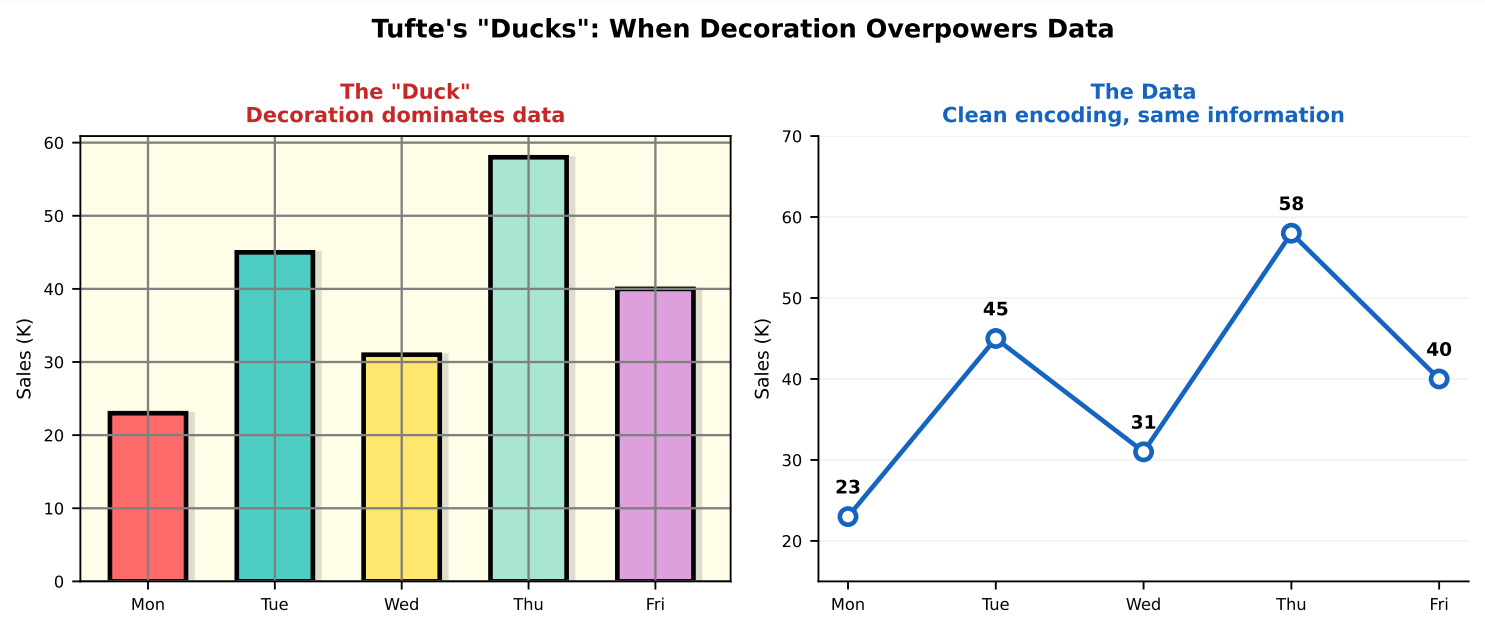
Moiré Vibration



Pro-Tip

Never use crosshatching to distinguish categories. Distinct hues at moderate saturation are perceptually superior and accessible.

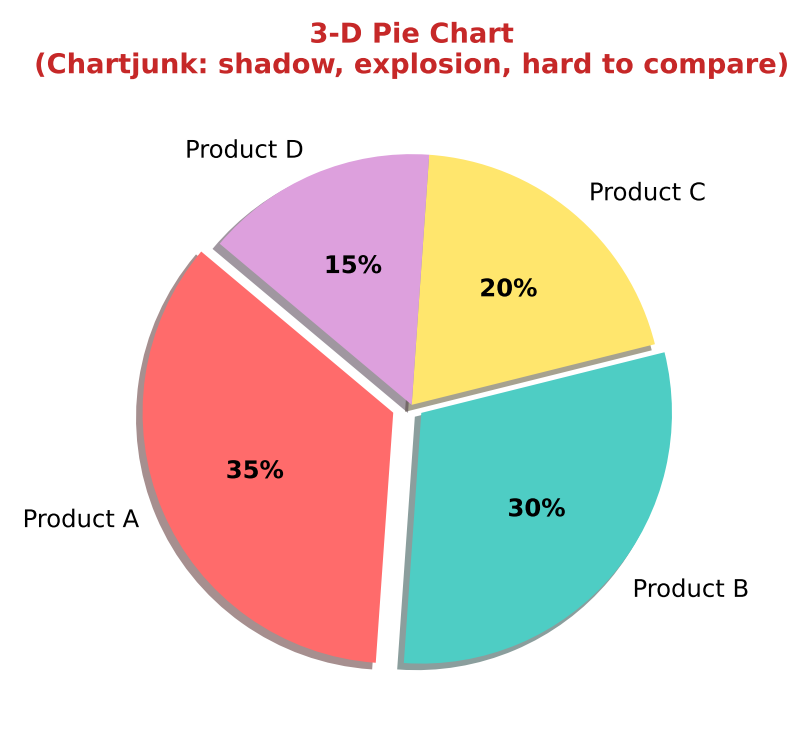
The Duck: Decoration vs. Data

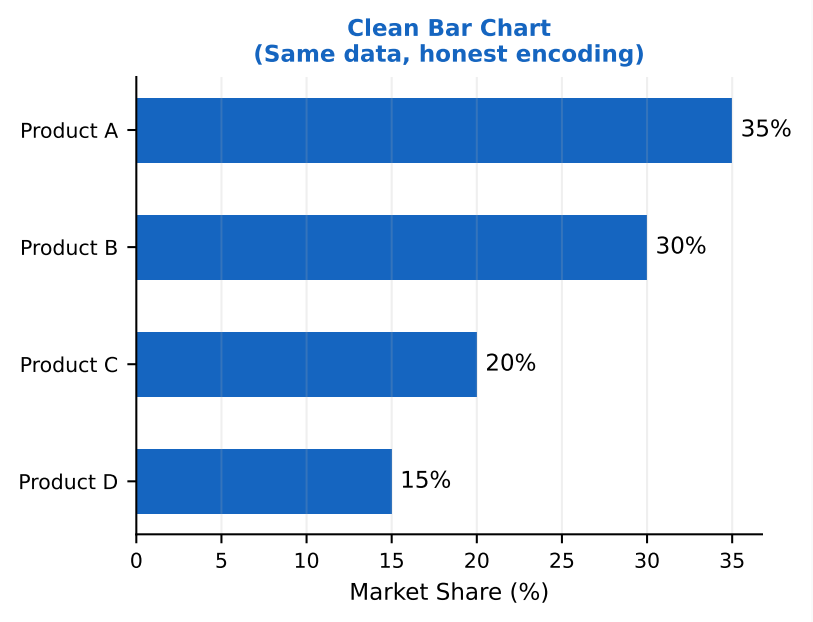


Data Insight

Tufte borrows "the duck" from architecture (Venturi et al., 1972): a building shaped like a duck to advertise a duck shop. In data graphics the decoration becomes the message, displacing the data.

The 3-D Pie: Chartjunk in Action



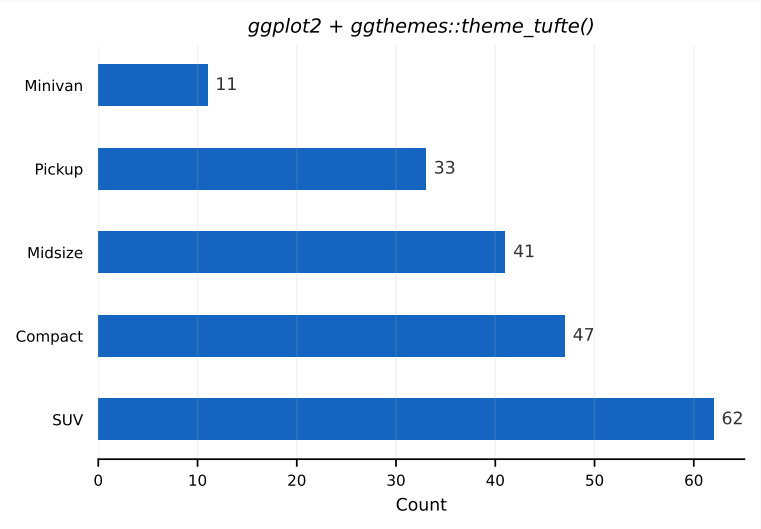


Problems with 3-D pies:

- Perspective distorts angular area (rear slices appear smaller)
- Shadow and explosion add non-data ink
- Humans compare lengths far better than angles

Removing Chartjunk in R

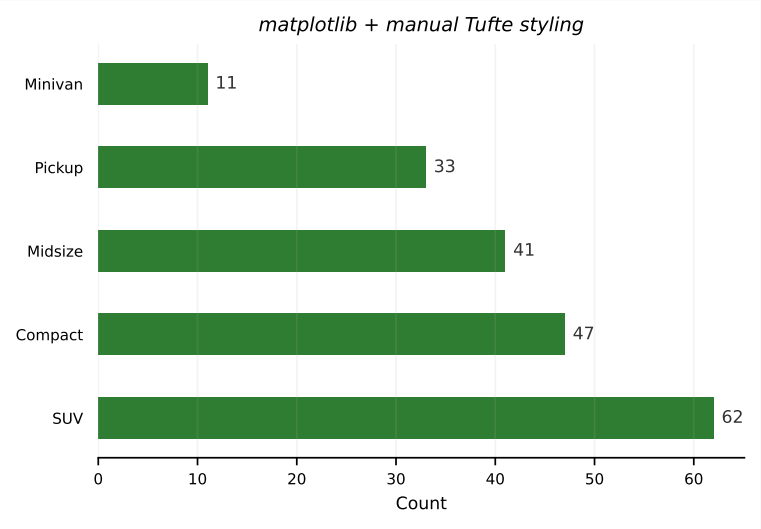
```r
# BEFORE: default bar with grid
ggplot(df, aes(x = cat, y = val)) +
  geom_bar(stat = "identity",
           fill = "skyblue",
           color = "black")
  # heavy gridlines by default

# AFTER: Tufte-style cleanup
ggplot(df, aes(x = fct_reorder(cat,
                val), y = val)) +
  geom_col(fill = "#1565C0",
           width = 0.6) +
  coord_flip() +
  theme_minimal() +
  theme(
    panel.grid.major.y =
      element_blank(),
    panel.grid.minor =
      element_blank()
  )
```



Key changes: flip to horizontal, remove y-grid, direct-label bars.

Removing Chartjunk in Python

```python
# BEFORE: default matplotlib bar
fig, ax = plt.subplots()
ax.bar(cats, vals)
# black border, grey background

# AFTER: systematic cleanup
fig, ax = plt.subplots()
ax.barh(cats, vals,
        color="#2E7D32", height
            =0.5)

# Remove 3 of 4 spines
for sp in ["top","right","left"]:
    ax.spines[sp].set_visible(False
        )

# Replace y-axis with direct labels
ax.tick_params(left=False)
for i, v in enumerate(vals):
    ax.text(v+0.8, i, str(v),
            va="center")

ax.grid(axis="x", alpha=0.15)
```



Python requires explicit spine removal; R's `theme_tufte()` handles it declaratively.

Graphical Integrity

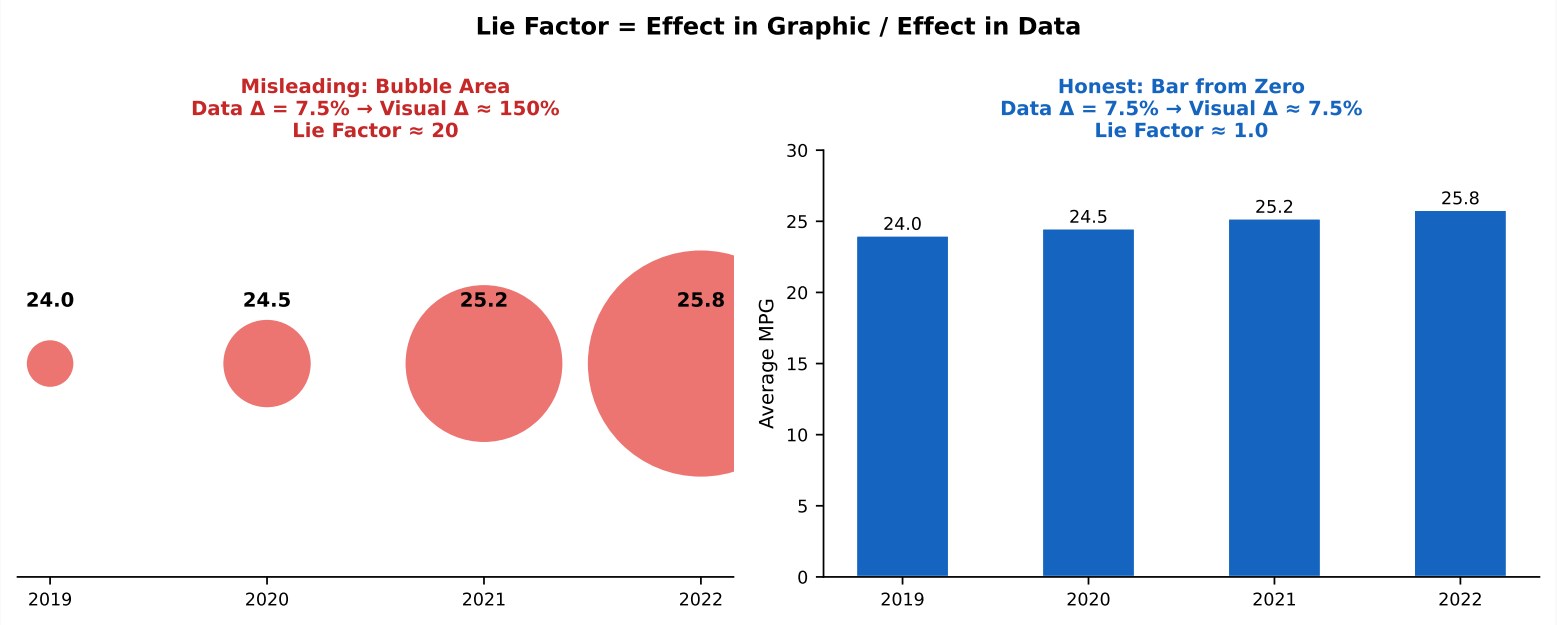
The Lie Factor

$$\text{Lie Factor} = \frac{\text{Size of effect in graphic}}{\text{Size of effect in data}}$$

- Lie factor $= 1.0$ means the visual change matches the data change
- Values outside $[0.95,\ 1.05]$ signal distortion
- Most common sources: truncated axes, area-encoded bars, perspective projections

Pro-Tip
Calculate the lie factor explicitly: measure the percentage change in the visual encoding (height, area, position) and divide by the percentage change in the data. Report it alongside your chart.

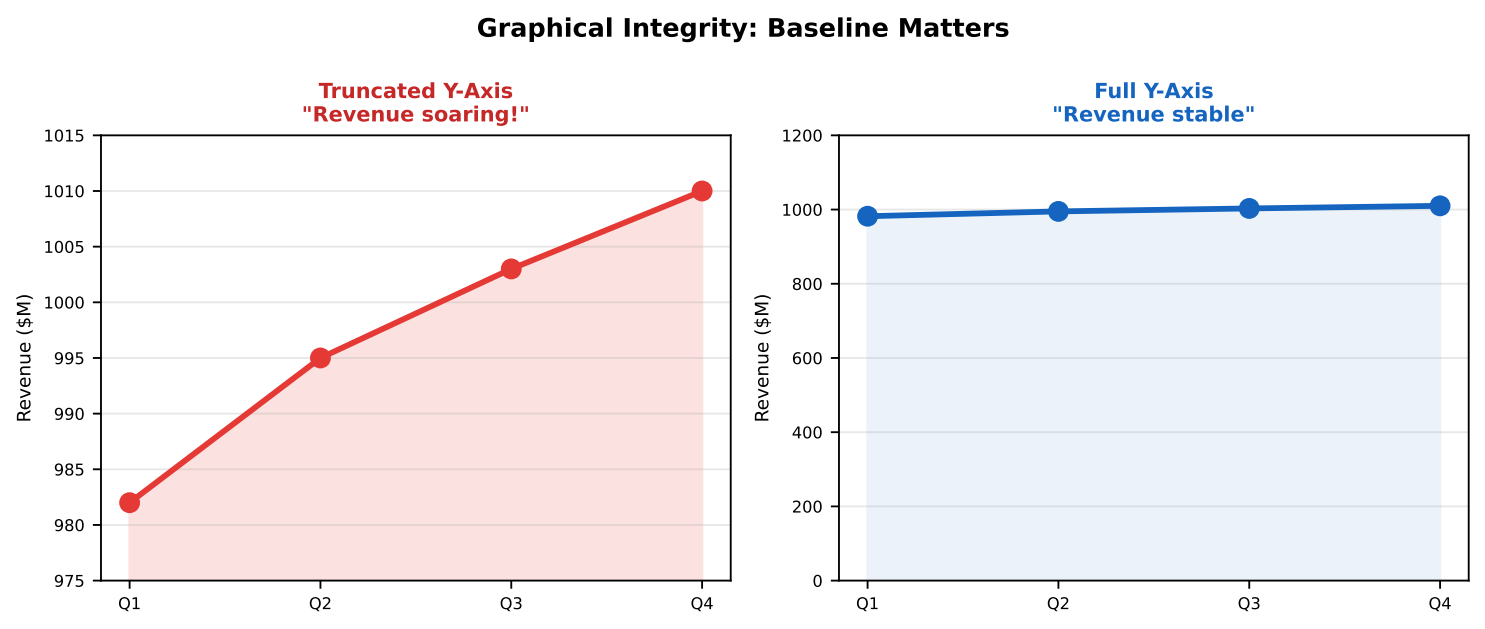
Lie Factor: Detailed Example



Data Insight

When bubble *area* encodes a 7.5% increase but the *visual* impression suggests 150% growth, the lie factor is $150/7.5 = 20$. Bar charts from zero keep the lie factor near 1.0.

Truncated Axes: The Most Common Lie



- A 2.8% revenue increase appears "dramatic" on a truncated axis
- The same data on a zero-based axis reveals stability
- Context determines when a non-zero baseline is legitimate (e.g., temperature anomalies)

Detecting Lie Factors in R

```r
# Compute lie factor for a chart
# where bar heights represent
    values

data_effect <- (new_val - old_val)
    /
                old_val * 100

# Measure visual effect:
# pixel height of new bar vs old
    bar
visual_effect <-
  (new_px - old_px) / old_px * 100

lie_factor <- visual_effect /
               data_effect

cat("Data change:", data_effect, "%
    ")
cat("Visual change:",
    visual_effect, "%")
cat("Lie factor:", lie_factor)

# Rule of thumb:
# 0.95 <= lie_factor <= 1.05
```

Worked example:

Metric	Value
Old value	\$982M
New value	\$1,010M
Data Δ	2.85%
Old bar (px)	10 px
New bar (px)	50 px
Visual Δ	400%
Lie Factor	**140.4**

Truncating the y-axis from 975 inflated a 2.85% change to 400%.

Fixing Lie Factors in Python

```python
import matplotlib.pyplot as plt

quarters = ["Q1","Q2","Q3","Q4"]
revenue = [982, 995, 1003, 1010]

fig, ax = plt.subplots()

# FIX 1: start y-axis from zero
ax.bar(quarters, revenue,
       color="#1565C0", width=0.5)
ax.set_ylim(0, 1200)

# FIX 2: add direct labels
for i, v in enumerate(revenue):
    ax.text(i, v + 15, f"${v}M",
            ha="center", fontsize
                =8)

# FIX 3: remove non-data ink
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
     False)

ax.set_ylabel("Revenue ($M)")
plt.tight_layout()
```

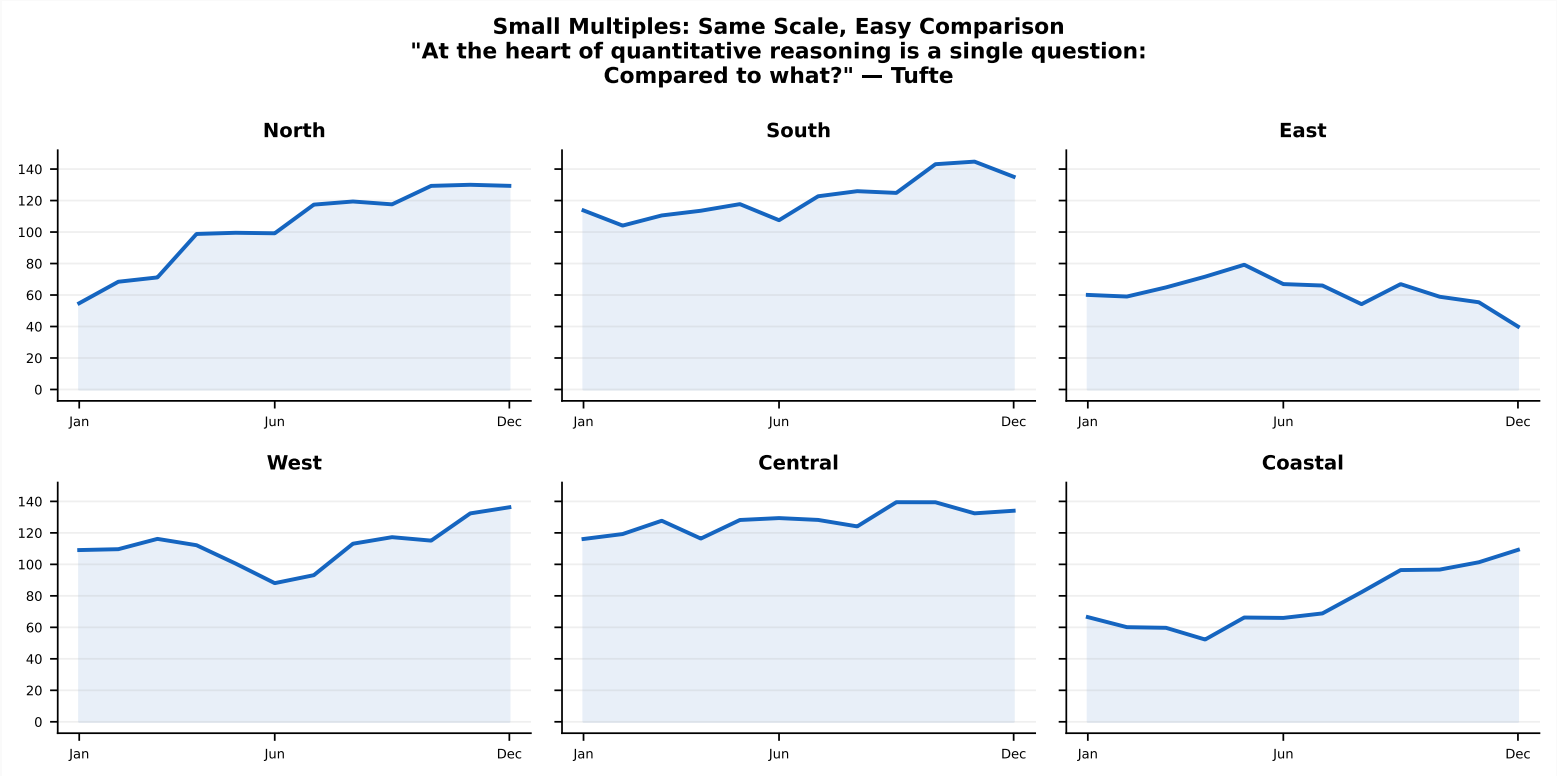
Three-step fix:

1. Set `ylim(0, max)` to anchor the baseline at zero
2. Add direct value labels so readers do not estimate from the axis
3. Remove non-data spines to maximise the data-ink ratio

Result: lie factor drops from $\sim$140 to $\sim$1.0.

Small Multiples & Sparklines

Small Multiples



Data Insight

By repeating the same design structure across panels, the viewer's eye is trained once and then compares freely. The design drops out; variation in the data emerges.

Small Multiples in R

```r
# facet_wrap creates small
    multiples
ggplot(df, aes(x = month,
               y = revenue)) +
  geom_line(color = "#1565C0",
            linewidth = 0.8) +
  geom_area(fill = "#1565C0",
            alpha = 0.08) +
  facet_wrap(~region, ncol = 3,
             scales = "fixed") +
  theme_minimal(base_size = 9) +
  theme(
    strip.text =
      element_text(face = "bold"),
    panel.grid.minor =
      element_blank()
  ) +
  labs(x = NULL, y = "Revenue ($K)"
       )
```

Design rules:

- Use `scales = "fixed"` so panels share the same y-axis
- Facet label replaces a legend
- Minimal grid: y-axis only
- Consistent colour across panels

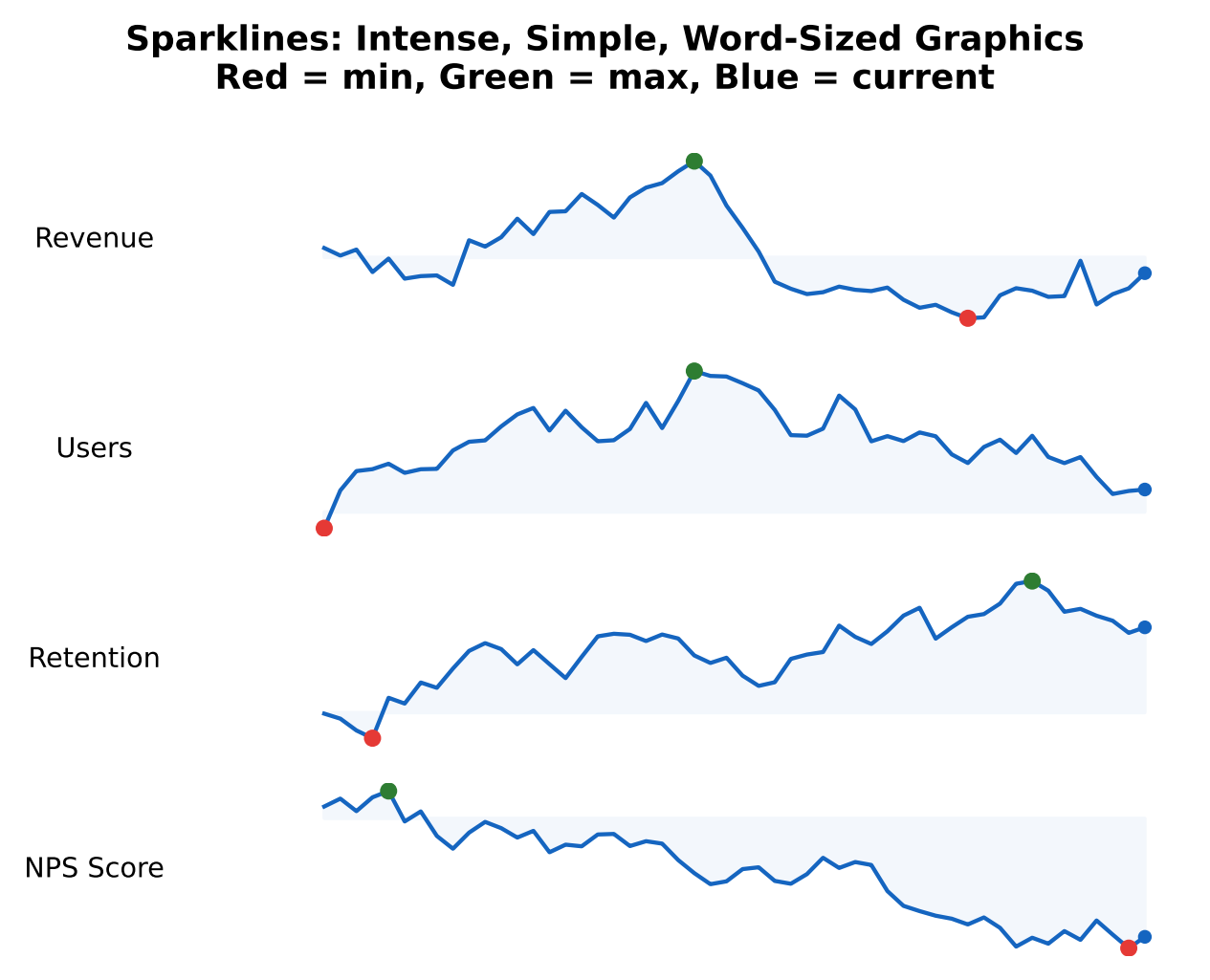
Small Multiples in Python

```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(
    2, 3, figsize=(10, 5),
    sharey=True  # fixed y-scale
)

for ax, region in zip(
    axes.flatten(), regions):

    ax.plot(months, data[region],
            color="#1565C0", lw
                =1.5)
    ax.fill_between(
        months, data[region],
        alpha=0.08, color="#1565C0"
            )
    ax.set_title(region,
                 fontsize=9,
                 fontweight="bold")

    # Tufte cleanup
    ax.spines["top"].set_visible(
        False)
    ax.spines["right"].set_visible(
        False)
```

Python equivalent:

- `sharey=True` ensures shared scale (like `scales="fixed"`)
- Loop over axes and data
- Spine removal must be explicit per subplot
- `tight_layout()` prevents label overlap

Sparklines: Word-Sized Graphics



Data Insight

Tufte introduced sparklines as "intense, simple, word-sized graphics" (2006, p. 7). They embed temporal context directly in text or tables,

Sparklines in R

```r
# Using ggplot2 for sparkline
    panels
library(ggplot2)

ggplot(weekly_data,
       aes(x = week, y = value)) +
  geom_line(color = "#1565C0",
            linewidth = 0.6) +
  # Mark endpoints
  geom_point(
    data = endpoints,
    aes(color = type),
    size = 2) +
  scale_color_manual(
    values = c(min = "#E53935",
               max = "#2E7D32",
               end = "#1565C0")) +
  facet_wrap(~metric, ncol = 1,
             scales = "free_y") +
  theme_void() +  # total erasure
  theme(strip.text =
    element_text(hjust = 0))
```

`theme_void()` erases:

- All axes and ticks
- All gridlines
- All background elements

Only the data line and three marker points remain: minimum (red), maximum (green), and current value (blue).

Alternative: `sparkline` package for inline HTML sparklines in RMarkdown/Quarto.

Sparklines in Python

```python
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(
    4, 1, figsize=(5, 4))

for ax, metric in zip(
    axes, metrics):
    data = series[metric]

    ax.plot(data, color="#1565C0",
            linewidth=1.0)

    # Mark min, max, endpoint
    ax.plot(np.argmin(data),
            data.min(), "o",
            color="#E53935", ms=4)
    ax.plot(np.argmax(data),
            data.max(), "o",
            color="#2E7D32", ms=4)

    # Strip ALL non-data elements
    ax.set_xticks([])
    ax.set_yticks([])
    for sp in ax.spines.values():
        sp.set_visible(False)
```

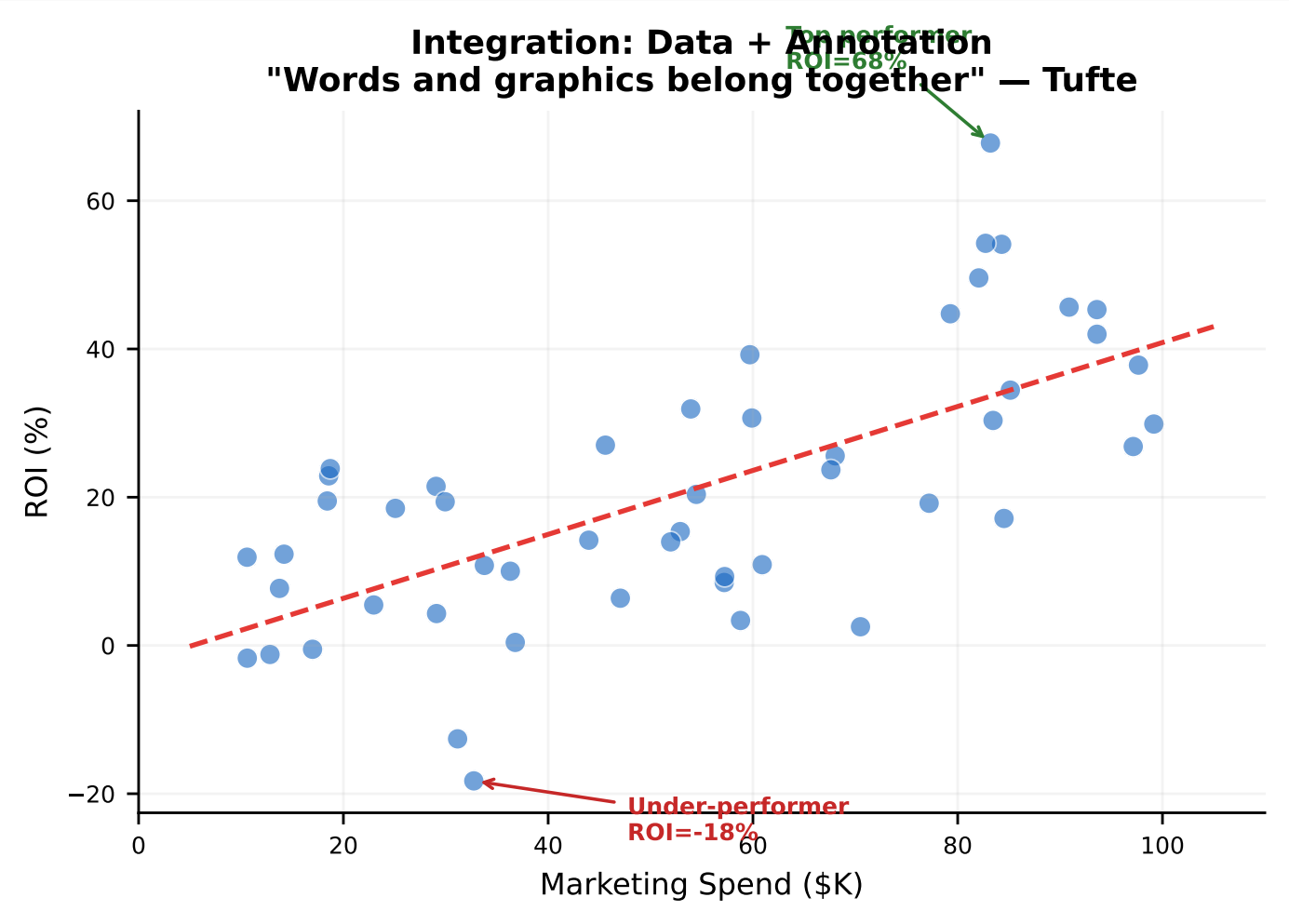
Design pattern:

1. Plot the line (data only)
2. Add exactly 3 marker points
3. Remove *all* spines, ticks, gridlines
4. Use `set_ylabel` with `rotation=0` for row labels

Data-ink ratio approaches 1.0: every pixel is either the trend line or a key data point.

Integration

Words and Graphics Belong Together



Data Insight

Tufte argues that segregating text from graphics is "a legacy of nineteenth-century engraving." Modern tools can seamlessly embed annotations, callouts, and contextual labels directly onto the data plane

Annotation in R

```r
ggplot(df, aes(x = spend,
               y = roi)) +
  geom_point(alpha = 0.5) +

  # Direct annotation
  annotate("text",
    x = 85, y = 65,
    label = "Top performer",
    fontface = "bold",
    color = "#2E7D32",
    size = 3) +

  # Arrow pointing to data
  annotate("segment",
    x = 83, xend = 78,
    y = 63, yend = 58,
    arrow = arrow(length =
      unit(0.2, "cm")),
    color = "#2E7D32") +

  # ggrepel for non-overlapping
  # geom_text_repel(aes(label))

  theme_minimal()
```

R annotation toolkit:

- `annotate("text")` for free-floating labels
- `annotate("segment")` for arrows and connectors
- `ggrepel` package for automatic label placement
- `ggtext` for markdown-formatted annotations

Annotation in Python

```python
fig, ax = plt.subplots()
ax.scatter(spend, roi, alpha=0.5)

# Annotate with arrow
ax.annotate(
    "Top performer\nROI=65%",
    xy=(78, 58),        # data point
    xytext=(85, 68),    # label pos
    fontsize=8,
    fontweight="bold",
    color="#2E7D32",
    arrowprops=dict(
        arrowstyle="->",
        color="#2E7D32",
        linewidth=1.2
    )
)

# Text box with background
ax.text(20, 55,
    "Quadrant analysis",
    fontsize=8,
    bbox=dict(
        boxstyle="round",
        facecolor="#E3F2FD",
        alpha=0.8))
```

Python annotation toolkit:

- `ax.annotate()` with `arrowprops` for arrows
- `ax.text()` with bbox for text boxes
- `adjustText` package for automatic repulsion
- `FancyArrowPatch` for custom arrow styles

Synthesis

Tufte’s Five Core Principles — Summary

Tufte's Five Core Principles

1. **Data-Ink Ratio**
   Maximise the share of ink devoted to data
2. **Chartjunk**
   Eliminate non-data ink that does not inform
3. **Lie Factor**
   Visual effect must match the data effect
4. **Small Multiples**
   Repeat a design to show variation across categories
5. **Integration**
   Words, numbers, and graphics belong together

The Checklist Applied

For every chart you produce, ask:

1. **Data-Ink**: Can I remove any element without losing information?
2. **Chartjunk**: Are there 3-D effects, moiré patterns, or decorative elements?
3. **Lie Factor**: Does the visual change proportionally match the data change?
4. **Comparison**: Would small multiples reveal more than a single cluttered panel?

Pro-Tip
"Above all else, show the data" does not mean "show *only* the data." Annotations, labels, and titles are data-adjacent ink that aids interpretation.

Key Takeaways

1. The **data-ink ratio** provides a quantitative target: maximise data ink, erase decoration.
2. **Chartjunk** — moiré, ducks, heavy grids — actively harms comprehension.
3. The **lie factor** is a measurable diagnostic: values outside $[0.95, 1.05]$ indicate deception.
4. **Small multiples** and **sparklines** are Tufte's most practical inventions — both available natively in R (`facet_wrap`, `theme_void`) and Python (`subplots`, spine removal).

Essential Readings for This Module

- Tufte, E. R. (1983). *The Visual Display of Quantitative Information*, Chapters 4–6 (Data-ink, Chartjunk, Data Density).
- Tufte, E. R. (2006). *Beautiful Evidence*, Chapter 2 (Sparklines).
- Wainer, H. (1984). "How to Display Data Badly," *The American Statistician*, 38(2), 137–147.
- Few, S. (2004). "Tapping the Power of Visual Perception," *Intelligent Enterprise*.

Next Module: Visual Perception & Pre-Attentive Attributes (W01-M03)